Domain and Range

My Definition

Facts & Characteristics

Domain

Examples

Non-examples

My Definition

Facts & Characteristics

Range

Examples

Non-examples

Contracts

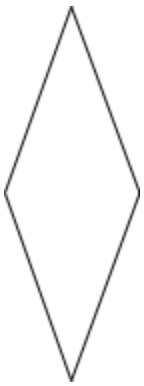
Contracts tell us how to use a function. For example: `ellipse :: (Number, Number, String, String) -> Image` tells us that the name of the function is `ellipse`, it takes four inputs (two Numbers and two Strings), and it evaluates to an `Image`. From the contract, we know `ellipse(50, 100, "solid", "teal")` will evaluate to an `Image`.

Name		Domain		Range
`# triangle`	`::`	`Number, String, String`	`->`	`Image`
`triangle(80, "solid", "darkgreen")`				
`# star`	`::`	`Number, String, String`	`->`	`Image`
`star(50, "solid", "teal")`				
`# circle`	`::`	`Number, String, String`	`->`	`Image`
`circle(30, "outline", "fuchsia")`				
`# square`	`::`	`Number, String, String`	`->`	`Image`
`square(10, "outline", "red")`				
`# rectangle`	`::`	`Number, Number, String, String`	`->`	`Image`
`rectangle(20, 80, "solid", "gold")`				
`# rhombus`	`::`	`Number, Number, String, String`	`->`	`Image`
`rhombus(20, 50, "solid", "turquoise")`				
`# ellipse`	`::`	`Number, Number, String, String`	`->`	`Image`
`ellipse(30, 70, "outline", "lightblue")`				
`# text`	`::`	`String, Number, String`	`->`	`Image`
`text("I'm thankful for…", 50, "green")`				
`# regular-polygon`	`::`	`Number, Number, String, String`	`->`	`Image`
`regular-polygon(8, 40, "solid", "red")`				
`#right-triangle`	`::`	`Number, Number, String, String`	`->`	`Image`

Using Contracts (continued)

Use the contracts to write expressions to generate images similar to those pictured.

`rhombus :: Number, Number, String, String -> Image`

	`rhombus(100, 140, "solid", "black")`
	`rhombus(100, 40, "outline", "black")`
Write an expression for a square (rotated) using `rhombus`!	`rhombus(100, 90, "solid", "black")`
What variable changes with the first Number?	*The length of each side*
What variable changes with the second Number?	*The top and bottom angle of the rhombus*

Triangle Contracts (SAS & ASA)

Type each expression (left) below into the editor and match it to the image it creates (right).

Expression			Image
`triangle-sas(120, 45, 70, "solid", "black")`	1	A	
`triangle-sas(120, 90, 70, "solid", "black")`	2	B	
`triangle-sas(120, 135, 70, "solid", "black")`	3	C	
`triangle-sas(70, 135, 120, "solid", "black")`	4	D	

Think about how you would describe each of the arguments that `triangle-sas` takes in to someone who'd never used the function before and annotate the contract below using descriptive variable names.

```
triangle-sas :: (
    ____________________side1____________________ :: Number,
    ____________________angle____________________ :: Number,
    ____________________side2____________________ :: Number,
    ____________________style____________________ :: String,
    ____________________color____________________ :: String
) -> Image
```

Add examples of each of the triangle functions we've explored to your contracts page.

If you have time, experiment with the `triangle-asa` function.

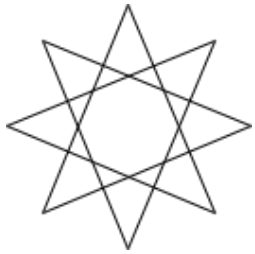
```
triangle-asa :: (

    left-angle :: Number,
    left-side :: Number,
    bottom-angle :: Number,
    style :: String
    color :: String
) -> Image
```

Star Polygon

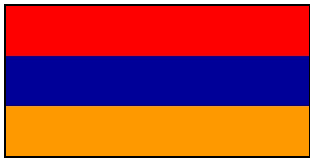
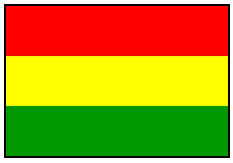
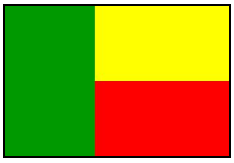
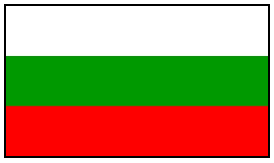
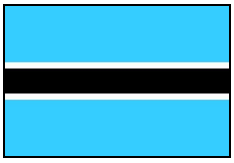
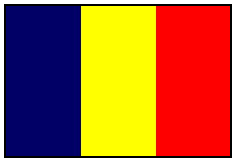
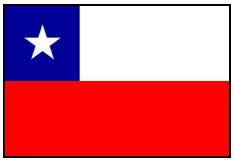
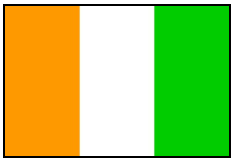
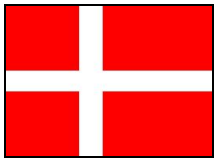
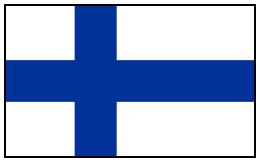
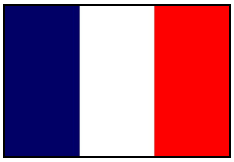
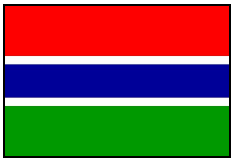
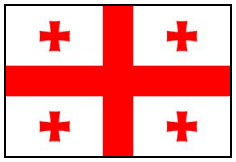
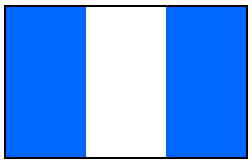
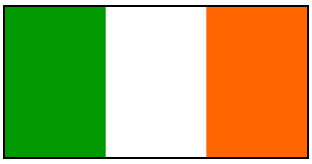
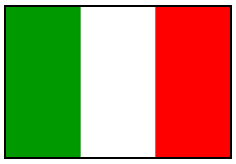
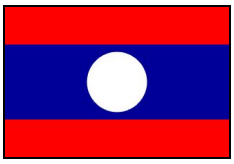
```
star-polygon :: (

    side-length :: Number,
    points-on-polygon :: Number,
    polygon-points-to-skip-between-star-points :: Number,
    shading-style  :: String,
    color :: String

) -> Image
```

Using the detailed contract above, write expressions to create each image below.

Then make two more star polygons of your choosing. Sketch them and write expressions to generate them.

	`star-polygon(50, 8, 3, "outline", "black")`
	`star-polygon(50, 8, 2, "solid", "black")`

These flags can all be created using the basic shape functions in addition to overlay, beside, and above.

 Armenia (1:2)	Belgium (13:15)	 Bolivia (15:22)	 Benin (2:3)
 Bulgaria (3:5)	 Botswana (2:3)	Burkina Faso (2:3)	Cameroon (2:3)
 Chad (2:3)	 Chile (2:3)	Costa Rica (3:5)	 Cote d'Ivoire (2:3)
 Denmark (28:37)	Estonia (7:11)	 Finland (11:18)	 France (2:3)
Gabon (3:4)	 The Gambia (2:3)	 Georgia (2:3)	Germany (3:5)
Ghana (2:3)	Greece (2:3)	 Guatemala (5:8)	Guinea-Bissau (1:2)
Hungary (1:2)	 Ireland (1:2)	 Italy (2:3)	 Laos (2:3)

Writing Examples from Purpose Statements 2

We've provided contracts and purpose statements to describe two different functions. Write examples for each of those functions.

Contract and Purpose Statement

Every contract has three parts…

half-image:: *String* -> *Image*

function name *Domain* *Range*

Consumes an image, and produces that image scaled to half its size.

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

half-image("*code.pyret.org/img/pyret-logo.png*") **is**

function name *input(s)*

scale(0.5, image-url("code.pyret.org/img/pyret-logo.png"))

what the function produces

half-image("*bootstrapworld.org/images/icon.png*") **is**

function name *input(s)*

scale(0.5, image-url("bootstrapworld.org/images/icon.png"))

what the function produces

end

Contract and Purpose Statement

Every contract has three parts…

product-squared:: *Number, Number* -> *Number*

function name *Domain* *Range*

Consumes two numbers and squares their product

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

product-squared(*5, 6*) **is** *num-sqr(5 * 6)*

function name *input(s)* *what the function produces*

product-squared(*10, 17*) **is** *num-sqr(10 * 17)*

function name *input(s)* *what the function produces*

end

Rocket Height Challenges

1) Can you make the rocket fly faster?

multiply `seconds` *by a number larger than 7:* $height(seconds) = 20 \times seconds$

2) Can you make the rocket fly slower?

multiply `seconds` *by a number smaller than 7:* $height(seconds) = 2 \times seconds$

3) Can you make the rocket sink down instead of fly up?

multiply `seconds` *by a negative number:* $height(seconds) = -7 \times seconds$

4) Can you make the rocket accelerate over time, so that it moves faster the longer it flies?

The formula for acceleration is quadratic: $height(seconds) = 7 \times seconds^2$.

5) Can you make the rocket blast off and then land again?

A parabola with a negative coefficient, shifted right by subtracting from the squared term and up by adding a constant:
$height(seconds) = -(seconds - 10)^2 + 100$

6) Can you make the rocket blast off, reach a maximum height of exactly 1000 meters, and then land?

In the previous solution, the constant we add is the square of the constant we subtract from $seconds$. *To reach a height of 1000:*
$height(seconds) = -(seconds - \sqrt{1000})^2 + 1000$

7) Can you make the rocket blast off, reach a maximum height of exactly 1000 meters, and then land after exactly 100 seconds?

A good challenge for a unit on quadratics! $height(seconds) = 0.4 \times (seconds)^2 + 40seconds$

8) Can you make the rocket fly to the edge of the the universe?

What's the inverse function of log? Exponential

Use any number for the base and seconds as the exponent: $height(seconds) = 7^{seconds}$

Blank Design Recipe

Directions:

Contract and Purpose Statement

Every contract has three parts…

_____ :: _____ -> _____
function name *Domain* *Range*

_____
what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

_____ (_____ "" _____) **is** _____
function name *input(s)* *what the function produces*

_____ (_____ "" _____) **is** _____
function name *input(s)* *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun _____ (_____):
function name *variable(s)*

_____
what the function does with those variable(s)

end

Directions:

Contract and Purpose Statement

Every contract has three parts…

_____ :: _____ -> _____
function name *Domain* *Range*

_____
what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

_____ (_____ "" _____) **is** _____
function name *input(s)* *what the function produces*

_____ (_____ "" _____) **is** _____
function name *input(s)* *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun _____ (_____):
function name *variable(s)*

_____
what the function does with those variable(s)

end

The Design Recipe (Restaurants)

Directions: Use the Design Recipe to write a function `split-tab` that takes in a cost and the number of people sharing the bill and splits the cost equally.

Contract and Purpose Statement

Every contract has three parts…

split-tab:: Number, Number -> Number
function name — Domain — Range

Consumes a cost and the # of people splitting, and produces each person's share
what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

split-tab(200, 10) **is** 200 / 10
function name — input(s) — what the function produces

split-tab(500, 5) **is** 500 / 5
function name — input(s) — what the function produces

end

Definition

Write the definition, giving variable names to all your input values…

fun split-tab(cost, people):
function name — variable(s)

cost / people
what the function does with those variable(s)

end

Directions: Use the Design Recipe to write a function `tip-calculator` that takes in the cost of a meal and returns the 15% tip for that meal.

Contract and Purpose Statement

Every contract has three parts…

tip-calculator:: Number -> Number
function name — Domain — Range

Consumes cost of a meal, and multiplies by 0.15 to produce the tip
what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

tip-calculator(10) **is** 0.15 * 10
function name — input(s) — what the function produces

tip-calculator(35) **is** 0.15 * 35
function name — input(s) — what the function produces

end

Definition

Write the definition, giving variable names to all your input values…

fun tip-calculator(cost):
function name — variable(s)

0.15 * cost
what the function does with those variable(s)

end

The Design Recipe (Direct Variation)

Directions: Use the Design Recipe to write a function `wage`, that takes in a number of hours worked and returns the amount a worker will get paid if their rate is $10.25/hr.

Contract and Purpose Statement

Every contract has three parts…

wage:: Number -> Number
function name — Domain — Range

Consumes a number of hours, multiplies it by $10.25 and returns that value
what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

wage(0) **is** 0 * 10.25
function name — input(s) — what the function produces

wage(30) **is** 30 * 10.25
function name — input(s) — what the function produces

end

Definition

Write the definition, giving variable names to all your input values…

fun wage(hours):
function name — variable(s)

hours * 10.25
what the function does with those variable(s)

end

Directions: On average, people burn about 11 calories/minute riding a bike. Use the Design Recipe to write a function `calories-burned` that takes in the number of minutes you bike and returns the number of calories burned. .

Contract and Purpose Statement

Every contract has three parts…

calories-burned:: Number -> Number
function name — Domain — Range

Consumes minutes biked, and multiplies by 11 to produce the calories.
what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

calories-burned(10) **is** 10 * 11
function name — input(s) — what the function produces

calories-burned(200) **is** 200 * 11
function name — input(s) — what the function produces

end

Definition

Write the definition, giving variable names to all your input values…

fun calories-burned(minutes):
function name — variable(s)

minutes * 11
what the function does with those variable(s)

end

The Design Recipe (Slope/Intercept 1)

Directions: For his birthday, James' family decided to open a savings account for him. He started with $50 and committed to adding $10 a week from his afterschool job teaching basketball to kindergartners. Use the Design Recipe to write a function `savings` that takes in the number of weeks since his birthday and calculates how much money he has saved.

Contract and Purpose Statement

Every contract has three parts…

```
# savings:: Number -> Number
```

function name — *Domain* — *Range*

```
# Consumes number of weeks, multiplies by 10 and returns the sum of the product and 50.
```

what does the function do?

Examples

Write some examples, then circle and label what changes…

```
examples:
  savings(10) is (10 * 10) + 50
  savings(17) is (10 * 17) + 50
end
```

function name — *input(s)* — *what the function produces*

Definition

Write the definition, giving variable names to all your input values…

```
fun savings(weeks):
  (10 * weeks) + 50
end
```

function name — *variable(s)* — *what the function does with those variable(s)*

Directions: Use the Design Recipe to write a function `moving` that takes in the days and number of miles driven and returns the cost of renting a truck. The truck is $45 per day and each driven mile is 15¢.

Contract and Purpose Statement

Every contract has three parts…

```
# moving:: Number, Number -> Number
```

function name — *Domain* — *Range*

```
# Consumes a number of days and multiplies it by $45, takes in a number of miles and multiplies it by $0.15, then adds the two
# products to produce the cost of moving
```

what does the function do?

Examples

Write some examples, then circle and label what changes…

```
examples:
  moving(1, 600) is (1 * 45) + (600 * 0.15)
  moving(3, 1500) is (3 * 45) + (1500 * 0.15)
end
```

function name — *input(s)* — *what the function produces*

Definition

Write the definition, giving variable names to all your input values…

```
fun moving(days, miles):
  (days * 45) + (miles * 0.15)
end
```

function name — *variable(s)* — *what the function does with those variable(s)*

The Design Recipe (Negative Slope/Intercept)

Directions: An Olympic pool holds 660,000 gallons of water. A fire hose can spray about 250 gallons per minute. Use the Design Recipe to write a function `pool` that takes in the number of minutes that have passed and calculates how much water is still needed to fill it.

Contract and Purpose Statement

Every contract has three parts…

```
# pool:: Number -> Number
```

function name — *Domain* — *Range*

```
# Consumes the number of minutes, multiplies them by 250 and subtracts the product from 660,000.
```

what does the function do?

Examples

Write some examples, then circle and label what changes…

```
examples:
  pool( 10 ) is 660000 - (10 * 250)
  pool( 40 ) is 660000 - (40 * 250)
end
```

function name — *input(s)* — *what the function produces*

Definition

Write the definition, giving variable names to all your input values…

```
fun pool( minutes ):
  660000 - (minutes * 250)
end
```

function name — *variable(s)* — *what the function does with those variable(s)*

Directions: The community arts fund awards a $1500 grant each month to support a new mural. They started with $50000 in their account. Use the Design Recipe to write a function `funds-available` that takes in the number of months and calculates how much money they have left.

Contract and Purpose Statement

Every contract has three parts…

```
# funds-available:: Number -> Number
```

function name — *Domain* — *Range*

```
# Consumes number of months, multiplies it by 1500, and subtracts the product from 50000
```

what does the function do?

Examples

Write some examples, then circle and label what changes…

```
examples:
  funds-available( 1 ) is 50000 - (1 * 1500)
  funds-available( 12 ) is 50000 - (12 * 1500)
end
```

function name — *input(s)* — *what the function produces*

Definition

Write the definition, giving variable names to all your input values…

```
fun funds-available( months ):
  50000 - (months * 1500)
end
```

function name — *variable(s)* — *what the function does with those variable(s)*

The Design Recipe (Geometry - Rectangles)

Directions: Use the Design Recipe to write a function `lawn-area` that takes in the length and width of a rectangular lawn and returns its area.

Contract and Purpose Statement

Every contract has three parts…

lawn-area:: *Number, Number* -> *Number*

function name *Domain* *Range*

Consumes length and width, and multiplies them and returns that value

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

lawn-area(*10, 20*) **is** *10 * 20*

function name *input(s)* *what the function produces*

lawn-area(*100, 300*) **is** *100 * 300*

function name *input(s)* *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun *lawn-area*(*length, width*):

function name *variable(s)*

*length * width*

what the function does with those variable(s)

end

Directions: Use the Design Recipe to write a function `rect-perimeter` that takes in the length and width of a rectangle and returns the perimeter of that rectangle.

Contract and Purpose Statement

Every contract has three parts…

rect-perimeter:: *Number, Number* -> *Number*

function name *Domain* *Range*

Consumes length and width, and doubles of their sum to produce the perimeter

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

rect-perimeter(*10, 20*) **is** *2 * (10 + 20)*

function name *input(s)* *what the function produces*

rect-perimeter(*200, 350*) **is** *2 * (200 + 350)*

function name *input(s)* *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun *rect-perimeter*(*length, width*):

function name *variable(s)*

*2 * (length + width)*

what the function does with those variable(s)

end

The Design Recipe (Geometry - Rectangular Prisms)

Directions: Use the Design Recipe to write a function `rectprism-vol` that takes in the length, width, and height of a rectangular prism and returns the Volume of a rectangular prism.

Contract and Purpose Statement

Every contract has three parts…

rectprism-vol:: *Number, Number, Number* -> *Number*

function name — *Domain* — *Range*

Consumes length, width, and height, and multiplies them to produce volume

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

`rectprism-vol(10, 20, 30) is 10 * (20 * 30)`

function name — *input(s)* — *what the function produces*

`rectprism-vol(100, 250, 350) is 100 * (250 * 350)`

function name — *input(s)* — *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun `rectprism-vol(length, width, height):`

function name — *variable(s)*

```
length * (width * height)
```

what the function does with those variable(s)

end

Directions: Use the Design Recipe to write a function `rect-prism-sa` that takes in the width, length and height of a rectangular prism and calculates its surface area (the sum of the areas of each of its six faces)

Contract and Purpose Statement

Every contract has three parts…

rect-prism-sa:: *Number, Number, Number* -> *Number*

function name — *Domain* — *Range*

*Consumes L, W, and H, and calculates the sum of 2*L*W, 2*L*H, and 2*W*H*

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

`rect-prism-sa(10, 20, 30) is`

function name — *input(s)*

```
((2 * (10 * 20)) + (2 * (10 * 30))) + (2 * (20 * 30))
```

what the function produces

`rect-prism-sa(5, 12, 13) is`

function name — *input(s)*

```
((2 * (5 * 12)) + (2 * (5 * 13))) + (2 * (12 * 13))
```

what the function produces

end

Definition

Write the definition, giving variable names to all your input values…

fun `rect-prism-sa(L, W, H):`

function name — *variable(s)*

```
((2 * (L * W)) + (2 * (L * H))) + (2 * (W * H))
```

what the function does with those variable(s)

end

The Design Recipe (Geometry - Circles)

Directions: Use the Design Recipe to write a function `circle-area-dec` that takes in a radius and uses the decimal approximation of pi (3.14) to return the area of the circle.

Contract and Purpose Statement

Every contract has three parts…

circle-area-dec:: *Number* -> *Number*

function name *Domain* *Range*

Consumes the radius, squares it, multiplies it by 3.14 and returns the area

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

`circle-area-dec(` *1* `)` **is** *3.14 * num-sqr(1)*

function name *input(s)* *what the function produces*

`circle-area-dec(` *3* `)` **is** *3.14 * num-sqr(3)*

function name *input(s)* *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun `circle-area-dec(` *radius* `):`

function name *variable(s)*

*3.14 * num-sqr(radius)*

what the function does with those variable(s)

end

Directions: Use the Design Recipe to write a function `circumference` that takes in a radius and uses the decimal approximation of pi (3.14) to return the circumference of the circle.

Contract and Purpose Statement

Every contract has three parts…

circumference:: *Number* -> *Number*

function name *Domain* *Range*

Consumes the radius, doubles it, multiplies it by 3.14 and returns the circumference.

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

`circumference(` *1* `)` **is** *(1 * 2) * 3.14*

function name *input(s)* *what the function produces*

`circumference(` *3* `)` **is** *(3 * 2) * 3.14*

function name *input(s)* *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun `circumference(` *radius* `):`

function name *variable(s)*

*(radius * 2) * 3.14*

what the function does with those variable(s)

end

The Design Recipe (Geometry - Cylinders)

Directions: Use the Design Recipe to write a function `circle-area` that takes in a radius and uses the fraction approximation of pi ($\frac{22}{7}$) to return the area of the circle.

Contract and Purpose Statement

Every contract has three parts…

```
# circle-area:: Number -> Number
```

function name — *Domain* — *Range*

```
# Consumes the radius, squares it, multiplies it by 22/7 and returns the area
```

what does the function do?

Examples

Write some examples, then circle and label what changes…

```
examples:
  circle-area( 1 ) is (22 / 7) * num-sqr(1)
  circle-area( 3 ) is (22 / 7) * num-sqr(3)
end
```

function name — *input(s)* — *what the function produces*

Definition

Write the definition, giving variable names to all your input values…

```
fun circle-area( radius ):
  (22 / 7) * num-sqr(radius)
end
```

function name — *variable(s)* — *what the function does with those variable(s)*

Directions: Use the Design Recipe to write a function `cylinder` that takes in a cylinder's radius and height and calculates its volume, making use of the function *circle-area*.

Contract and Purpose Statement

Every contract has three parts…

```
# cylinder:: Number, Number -> Number
```

function name — *Domain* — *Range*

```
# Consumes radius and height, and returns the product of the height and the area of the circle calculated using the radius and the
# function circle-area.
```

what does the function do?

Examples

Write some examples, then circle and label what changes…

```
examples:
  cylinder( 10, 20 ) is 20 * circle-area(10)
  cylinder( 200, 350 ) is 350 * circle-area(200)
end
```

function name — *input(s)* — *what the function produces*

Definition

Write the definition, giving variable names to all your input values…

```
fun cylinder( radius, height ):
  height * circle-area(radius)
end
```

function name — *variable(s)* — *what the function does with those variable(s)*

The Design Recipe (Breaking Even)

Directions: The Swamp in the City Festival is ordering t-shirts. The production cost is $75 to set up the silk screen and $9 per shirt. Use the Design Recipe to write a function `min-shirt-price` that takes in the number of shirts to be ordered, *n*, and returns the minimum amount the festival should charge for the shirts in order to break even. (Assume that they will sell all of the shirts.)

Contract and Purpose Statement

Every contract has three parts…

#min-shirt-price:: *Number* -> *Number*

function name *Domain* *Range*

#Consumes number of shirts, multiplies it by nine, adds 75, and then divides the sum by that number.

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

`min-shirt-price(10)` **is** `((9 * 10) + 75) / 10`

function name *input(s)* *what the function produces*

`min-shirt-price(100)` **is** `((9 * 100) + 75) / 100`

function name *input(s)* *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun `min-shirt-price(length, width):`

function name *variable(s)*

`((9 * n) + 75) / n`

what the function does with those variable(s)

end

The Design Recipe (Marquee & Cubing)

Directions: Use the Design Recipe to write a function `marquee` that takes in a message and returns that message in large gold letters.

Contract and Purpose Statement

Every contract has three parts…

marquee:: *String* -> *Image*

function name *Domain* *Range*

Consumes a message and returns an image of it in large gold letters

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

marquee(*"Hooray!"*) **is** *text("Hooray!", 70, "gold")*

function name *input(s)* *what the function produces*

marquee(*"It works"*) **is** *text("It works", 70, "gold")*

function name *input(s)* *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun *marquee*(*message*):

function name *variable(s)*

text(message, 70, "gold")

what the function does with those variable(s)

end

Directions: Use the Design Recipe to write a function `num-cube` that takes in a number and returns the cube of that number.

Contract and Purpose Statement

Every contract has three parts…

num-cube:: *Number* -> *Number*

function name *Domain* *Range*

Consumes a number, cubes it, and returns that value.

what does the function do?

Examples

Write some examples, then circle and label what changes…

examples:

num-cube(*1*) **is** *1 * (1 * 1)*

function name *input(s)* *what the function produces*

num-cube(*3*) **is** *3 * (3 * 3)*

function name *input(s)* *what the function produces*

end

Definition

Write the definition, giving variable names to all your input values…

fun *num-cube*(*n*):

function name *variable(s)*

*n * (n * n)*

what the function does with those variable(s)

end

Vocabulary Practice

Below is a new structure definition:

```
data MediaType:
  | book(
      title :: String,
      author :: String,
      pubyear :: Number)
end

# an example book:
book1 = book("1984", "Orwell", 1949)
```

Fill in the blanks below with the vocabulary term that applies to each name. Here are the terms to choose from:

contract	example
header	field
data type	instance
constructor	data block
name	purpose

`author` is a _____

`book` is a _____

`MediaType` is a _____

`book1` is a _____

`title` is a _____

`data` … **`end`** is a _____